

소프트웨어공학

소프트웨어 위험관리와 유지보수

- 1 소프트웨어 위험 관리
- 2 유지보수의 개념과 종류
- 3 유지보수의 문제점
- 4 신뢰도 측정

01

소프트웨어 위험관리

- 위험 관리의 절차
 - 위험 관리의 절차는 위험 식별, 위험 분석 및 평가, 위험 관리 계획, 위험 감시 및 조치 순이다.

위험 식별
(risk identification)

알려지거나 예측 가능한 위험 요소를 파악하는 작업으로,
위험 항목 점검 목록을 작성하여 활용한다

- 위험 관리의 절차
 - 위험 분석 및 평가
 - 위험 분석은 프로젝트에 내재한 위험 요소를 인식하고 그 영향을 분석하는 활동으로, 위험 추산(risk estimation) 작업을 통해 수행된다.
 - 가능한 모든 위험 요소와 영향 분석하여 의사결정에 반영한다.
 - 위험 요소에 대해 효과적이지 못한 관리는 프로젝트를 실패하는 결과도 가져올 수 있다.
 - 위험 추산을 위해 다음과 같은 위험표(risk table)를 작성하여 활용한다.

01

소프트웨어 위험관리

- 위험 관리의 절차

위험 내용	위험 범주 (종류)	발생 확률	영향력	위험 감시 및 조치
기술력이 부족하다.	BU	60%	2	
제품 규모의 추적이 낮 다.	PS	30%	3	
⋮	⋮	⋮	⋮	

- ① 위험 항목 점검 목록을 참조하여 위험 내용을 나열한다.
- ② 해당 위험이 속하는 범주(종류)를 기입한다.
(PS : 제품 사이즈 위험, BU : 비즈니스 위 험 등)
- ③ 위험 내용이 발생될 확률을 기입한다.
- ④ 위험 내용이 실제 발생될 경우 영향력을 기입한다.
(1 : 파멸적, 2 : 결정적, 3 : 중요하지 않음, 4 : 무시)
- ⑤ 위험 감시 및 조치 사항을 기입한다.

01

소프트웨어 위험관리

- 위험 관리의 절차
 - 위험 관리 계획
 - 위험을 예방하고 위험 발생 시 대책을 준비하며 문서화하는 작업이다.
 - 위험 감시 및 조치
 - 위험 내용을 항상 감시하고 실제 발생 시 조치하는 것으로 다음과 같은 전략을 사용한다.

위험 회피 (risk svoidance)	위험 관리에 대한 최상의 전략으로 위험이 발생할 것을 예상하고 회피하는 것이다.
위험 감시 (risk monitoring)	위험 요소 징후들에 대하여 계속적으로 인지하는 것이다.
위험 관리(risk management) 및 비상 계획(contingency plan) 수립	위험 회피 전략이 실패할 경우 위험에 대해 관리하고 대비책과 비상 계획을 세운다.

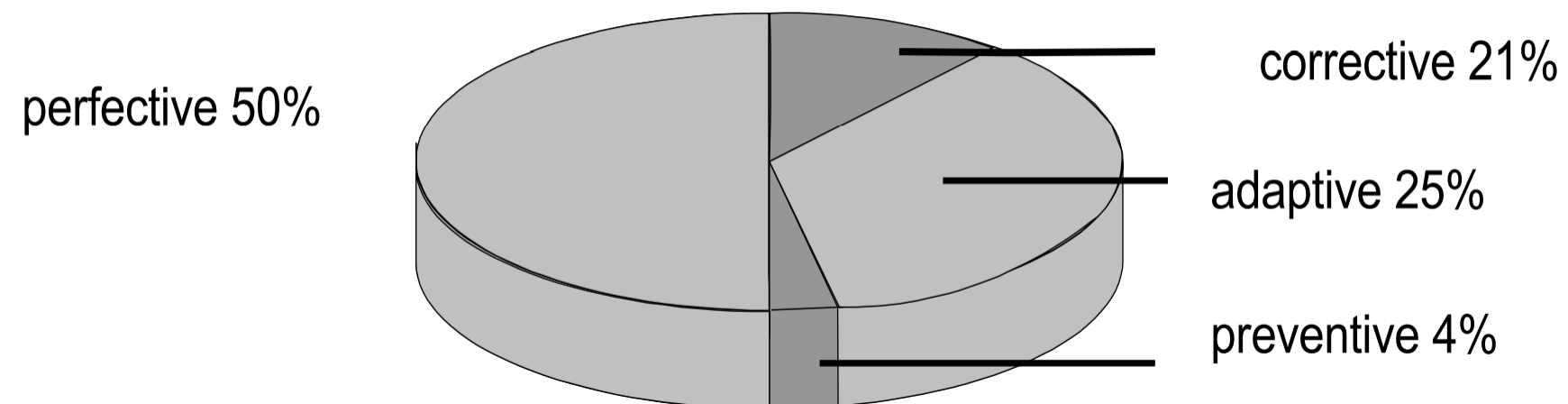
02

유지보수의 개념과 종류

유지보수 (maintenance)

개발된 소프트웨어의 품질을 항상 최상의 상태로 유지하기 위한 것으로, 소프트웨어 개발 단계 중 가장 많은 노력과 비용이 투입되는 단계이다.

- 유지보수는 소프트웨어가 사용자에게 인수되어 설치된 후 발생하는 모든 공학적 작업이다.
- 소프트웨어 유지보수를 용이하게 하려면 시험 용이성, 이해성, 수정 용이성, 이식성 등이 고려되어야 한다.
- 유지보수는 수정(corrective) 보수, 적응(adaptive) 보수, 완전화(perfective) 보수, 예방(preventive) 보수 활동으로 구분할 수 있고, 이 활동을 통해 소프트웨어의 수명을 연장 시키는 작업이다.



유지보수의 개념과 종류

- 유지보수의 분류

수정 보수	시스템을 운영하면서 검사 단계에서 발견하지 못한 잠재적인 오류를 찾아 수정하는 활동으로 오류의 수정과 진단을 포함
적응 보수	- 소프트웨어의 수정 기간 중에 발생하는 환경의 변화(하드웨어, 운영체제 등)를 기존의 소프트웨어에 반영하기 위하여 수행하는 활동 - 운영체제나 컴파일러와 같은 프로그래밍 환경의 변화와 주변장치 또는 다른 시스템 요소가 향상되거나 변경될 때 대처할 수 있는 유지보수 활동
완전화 보수	- 소프트웨어 본래 기능에 새로운 기능을 추가하거나 성능을 개선하기 위해 소프트웨어를 확장시키는 활동 - 유지보수 활동 중 가장 큰 업무 및 비용을 차지하는 활동
예방 보수	- 미래에 유지보수를 용이하게 하거나 기능을 향상시키기 위해 소프트웨어를 변경하는 활동 - 예방 유지보수는 소프트웨어 재공학이라고도 함

유지보수의 개념과 종류

- 유지보수의 과정
 - 소프트웨어 유지보수의 과정은 유지보수 요구, 현 시스템에 대한 이해, 수정 및 시험 순으로 반복해서 일어난다.
 - ① 유지보수 요구 : 유지보수 활동이 필요한 경우 이에 대한 요구를 수행한다.
 - ② 현 시스템에 대한 이해 : 요구된 유지보수 활동을 수행하기 위해서는 먼저 현재 시스템에 대한 이해를 필요로 하며, 현재 시스템에 대해 이해하지 못한 상태에서 유지보수 활동을 수행하면 오류가 발생할 수 있다.
 - ③ 수정 및 시험 : 유지보수 활동을 수행하고, 유지보수 활동을 수행한 후에도 정상적으로 기능이 수행되는지를 확인하기 위해 테스트한다.

03

유지보수의 문제점

- 다른 사람이 개발한 소프트웨어는 이해하기가 어려울 뿐만 아니라 소프트웨어 개발자 들의 이직률이 높기 때문에 개발된 소프트웨어에 대한 전문적인 설명을 들을 수가 없다.
- 소프트웨어에 대한 변경이 자주 발생하므로 변경된 내용을 문서화하지 않을 경우 추적 하기 어렵다.
- 소프트웨어에 대한 적절한 문서가 없거나 문서의 질이 형편없을 수도 있다.
- 대부분의 소프트웨어는 변경 가능하도록 설계되지 않는다.

03

유지보수의 문제점

- 유지보수의 부작용

- 유지보수 활동을 통해 예기치 못한 부작용이 발생할 수 있으며 그 종류는 다음과 같다.

코딩 부작용	코딩 내용의 변경으로 인해 발생하는 부작용
자료 부작용	자료나 자료 구조의 변경으로 인해 발생하는 부작용
문서화 부작용	자료 코드에 대한 변경이 설계문서나 사용자가 사용하는 매뉴얼에 적용되지 않을 때 발생하는 부작용

- 외계인 코드

외계인 코드 (alien code)	• 아주 오래 전에 개발되어 유지보수 작업이 매우 어려운 프로그램을 의미한다. • 일반적으로 15년 전 또는 그 전에 개발된 프로그램을 의미하며, 문서화 (documentation)를 철저하게 해두면 외계인 코드를 방지할 수 있다.
------------------------	---

유지보수의 문제점

- 유지보수의 문제들의 주요원인
 - 소프트웨어는 다른 제품과는 달리 무형이므로 자유로이 변경할 수 있다고 생각하는 경우가 많다.
 - 이 때문에 불필요한 변경의뢰가 사용자로부터 생기게 된다.
 - 그것은 사용자들이 소프트웨어는 완전한 블랙박스이며 실체가 없는 것이라고 생각하기 때문이다.
 - 유지보수에 관한 요구정의를 명확히 표현하기 위한 기법이 없다.
 - 신규개발의 소프트웨어와 마찬가지로 요구 정의가 불명확함으로 여러 가지 충돌이 발생한다.

- 유지보수의 문제들의 주요원인
 - 소프트웨어 작성공정이 장시간에 걸쳐 고정된 요구를 전제로 하기 때문에 아무래도 본 가동 후 금방 변경의뢰가 발생하게 된다.
 - 특히 대규모적인 소프트웨어일수록 이 경향이 커지게 된다.
 - 개발측의 버그(bug)와 사용자측의 요구변경의 구별이 많다.
 - 이 때문에 보수비용의 부담이 어느쪽이 되느냐에 의해 충돌이 일어나기 쉽다.
 - 체계화된 유지보수 지원도구가 거의 개발되어 있지 않고, 부분적인 지원 도구 정도 밖에 없다.
 - 이 때문에 보수공정의 생산성, 신뢰성이 매우 떨어지게 된다.

유지보수의 문제점

- 유지보수의 문제점들의 주요원인
 - 소프트웨어 설계 단계에서 보수를 고려한 방법론, 기법이 제안되어 있지 않다.
 - 보수를 생각하면, 소프트웨어의 이해용이성에 대한 기준이 설계상에서의 포인트가 되지만 효과적인 대책이 없는 상태이다.
 - 유지보수를 수행하기 위하여 필요한 문서가 준비되지 않는 상태에 있다.
 - 개발시에 작성된 문서가 그래도 보수용 자료로서 대신 사용되고 있는 것에 불과하다.
 - 소프트웨어의 규모가 커질수록 문서가 잘 정리되지 못한다.
 - 소프트웨어 규모의 증대에 의하여 문서 사이의 인터페이스가 복잡하게 되며, 관리하기가 벅찬 문서 양이 된다.

유지보수의 문제점

- 유지보수의 문제들의 주요원인
 - 유지보수 공정 가운데에서 문서의 품질이 떨어지는 경향이 많다.
 - 그것은 원시 프로그램의 변경을 수행한 후에 문서의 수정까지 손을 쓰지 못한 상태가 된다.
 - 이 결과로 최신의 원시 프로그램과 적합성이 떨어지는 문서가 남게 된다.
 - 유지보수 작업을 계속해 가면 개발단계에서는 구조화되어 알기가 쉽던 소프트웨어도 서서히 복잡하게 되어 버린다.
 - 프로그램의 구성 및 문서도 이와 같은 경향이 되어서 소프트웨어의 품질이 상대적으로 떨어지게 된다.

유지보수의 문제점

- 유지보수의 문제들의 주요원인
 - 소프트웨어의 수정에 파급효과가 생겨 그것에 의해 잠재적인 에러가 축적되기 쉽다.
 - 파급효과를 줄이기 위한 유지보수기법이 제안되어 있지 않다.
 - 타인이 작성한 소프트웨어를 다시 고치는 것은 자신이 작성한 것을 고치는 것보다 곤란하며, 잘못을 유발할 가능성이 높다.
 - 소프트웨어 개발공정에서 생겨난 작성자의 생각이나 사상까지 이해하지 않으면 소프트웨어의 전체가 파악되지 못하기 때문이다.
 - 개발에서부터 유지보수로의 인계시에 전제사항이 불투명하게 되기 쉽다.
 - 특히, 개발담당자와 유지보수담당자가 서로 다른 경우에 문제가 발생하기 쉽다.

유지보수의 문제점

- 유지보수의 문제들의 주요원인
 - 유지보수의 이력, 경과 정보가 거의 관리되어 있지 않다.
 - 이 때문에 유지보수요원을 교체할 때에 대응이 되지 않는다든지, 소프트웨어 유지보수에 일관성 있는 체제가 되지 못한다.
 - 유지보수작업의 견적 예측이 정확히 수행되지 못한다.
 - 유지보수에 드는 경비와 노력이 예정대로 되지 않으므로 유지보수작업을 관리하기가 어렵다.
 - 유지보수작업의 관리가 어렵다.
 - 유지보수작업은 그 해당되는 사람 이외에는 이해되지 않는 경우가 많다.
 - 변경의뢰에 의한 유지보수의 납기는 개발보다 짧다.
 - 또한, 유지보수에 대한 스케줄도 설정하기 어렵다.

유지보수의 문제점

- 유지보수의 문제들의 주요원인
 - 유지보수 요원의 작업에 대한 성취도가 저하하는 경우가 있다.
 - 창조성이 없는 작업 내용이나 타인의 생각을 이해하여 그것에 맞추는 수밖에 없으면, 자기표현의 장이 설정될 수 없다.
 - 유지보수 요원으로서 경험이 적은 신인 등이 배치되는 경우가 많다.
 - 유지보수를 실시 하기 위해서는 담당의 경험과 지식이 요구되지만 유지보수 작업 자체가 쉽게 받아 들여져 버리는 결과로 이러한 현상이 생기게 된다.
 - 유지보수요원의 배치전환은 매우 어렵다.
 - 유지보수공정 자체는 상당한 수년 사이에 계속적으로 지속되기 때문에 배정된 사람이 그대로 유지보수작업 하기가 어렵다.
 - 유지보수하고 있는 노하우가 그 사람밖에 축적되어 있지 않는 것도 원인이라고 할 수 있다.

유지보수의 문제점

- 유지보수의 문제들의 주요원인
 - 유지보수작업의 노동형태가 과로하기 쉽다.
 - 납기가 짧고, 낮에는 본래의 시스템이 가동되고 있기 때문에 야간에 소프트웨어의 변경을 하여야 하며, 잔업의 증가를 초래한다.
 - 업계 전체가 소프트웨어의 유지보수에 대한 중요성의 관심이 낮다.
 - 소프트웨어의 설계, 개발, 테스트 등의 테마에만 관심이 집중되어 있어서 유지보수에 관한 연구나 실용화에 어프로치가 미흡하다.

- MTBF
 - 하드웨어적인 신뢰도를 기반으로 MTBF를 이용하여 소프트웨어적인 신뢰성을 산출할 수 있다.
 - MTBF(Mean Time Between Failure) : 평균 고장 간격
 - 시스템이 고장난 후부터 다음 고장이 날때까지의 시간
 - $MTBF = MTTF + MTT$
 - MTTF(Mean Time To Failure): 평균 가동 시간
 - 시스템의 사용 시점부터 고장이 발생할 때까지의 가동 시간 평균
 - $MTTF = (가동시간1 + 가동시간2 + \dots + 가동시간n) / n$

04

신뢰도 측정

- MTBF
 - MTTR(Mean Time To Repair): 평균 수리 시간
 - 시스템에 고장이 발생하여 가동하지 못한 시간들의 평균
 - $MTTR = (고장시간1 + 고장시간2 + \dots + 고장시간n) / n$
 - 가용성(신뢰도)측정 : 정상적으로 가동된 시간의 비율
 - $MTBF / (MTBF + MTTR) \times 100\% = MTTF / (MTTF + MTTR) \times 100\%$
 - $MTTF / MTBF \times 100\%$

04

신뢰도 측정

- MTBF



- $MTTF = (\text{가동시간1} + \text{가동시간2} + \dots + \text{가동시간n})/n = (8+9+4)/3=7$
- $MTTR = (\text{고장시간1} + \text{고장시간2} + \dots + \text{고장시간n})/n = (2+4+1)/3=2.3$
- $MTBF = MTTF + MTTR = 7+2.3 = 9.3$
- $\text{가용성(신뢰도)} = MTTF / MTBF \times 100\% = 7/9.3 \times 100 = 0.75$

04

신뢰도 측정

- 신뢰도 요소(RAS)
 - 신뢰도(Reliability): 시스템 또는 각 구성요소가 정해진 조건으로 의도하는 기간 중에 소정의 기능을 완수하는 능력
 - 가용도(Availability): 어느 특정 시점에서 소정의 기능을 완수하고 있는 비율
 - 보전도(Serviceability): 시스템 또는 각 구성요소가 장애가 발생했을때 회복을 위한 수리의 간편도